

MANUAL DE DESPLIEGUE Y CONFIGURACIÓN DE SERVIDOR (DEPLOY)

Proyecto: Sistema de Gestión Integral (SGI) para Industria Metalúrgica

1. DATOS GENERALES DEL PROYECTO

- **Integrantes:** Anderson Sanchez, Agustina Heck
- **Curso:** 6to 2da C
- **Especialidad:** Computación / Programación
- **Institución:** E.T. N°32 "Gral. José de San Martín"
- **Año Lectivo:** 2025

1.2. Descripción de la Problemática El presente manual documenta el proceso de puesta en producción de una solución digital diseñada para optimizar los procesos operativos de una PyME del sector metalúrgico. La problemática central abordada es la falta de centralización de la información en las áreas de inventario, taller y ventas, lo que derivaba en una gestión ineficiente de insumos y falta de trazabilidad en los pedidos de manufactura.

1.3. Objetivo del Manual El objetivo primordial de este documento es servir como guía técnica detallada para el despliegue de la aplicación web. Describe la infraestructura utilizada (arquitectura en la nube), la configuración de la base de datos distribuida y los procedimientos de integración continua necesarios para garantizar que el sistema sea accesible de forma segura a través de internet.

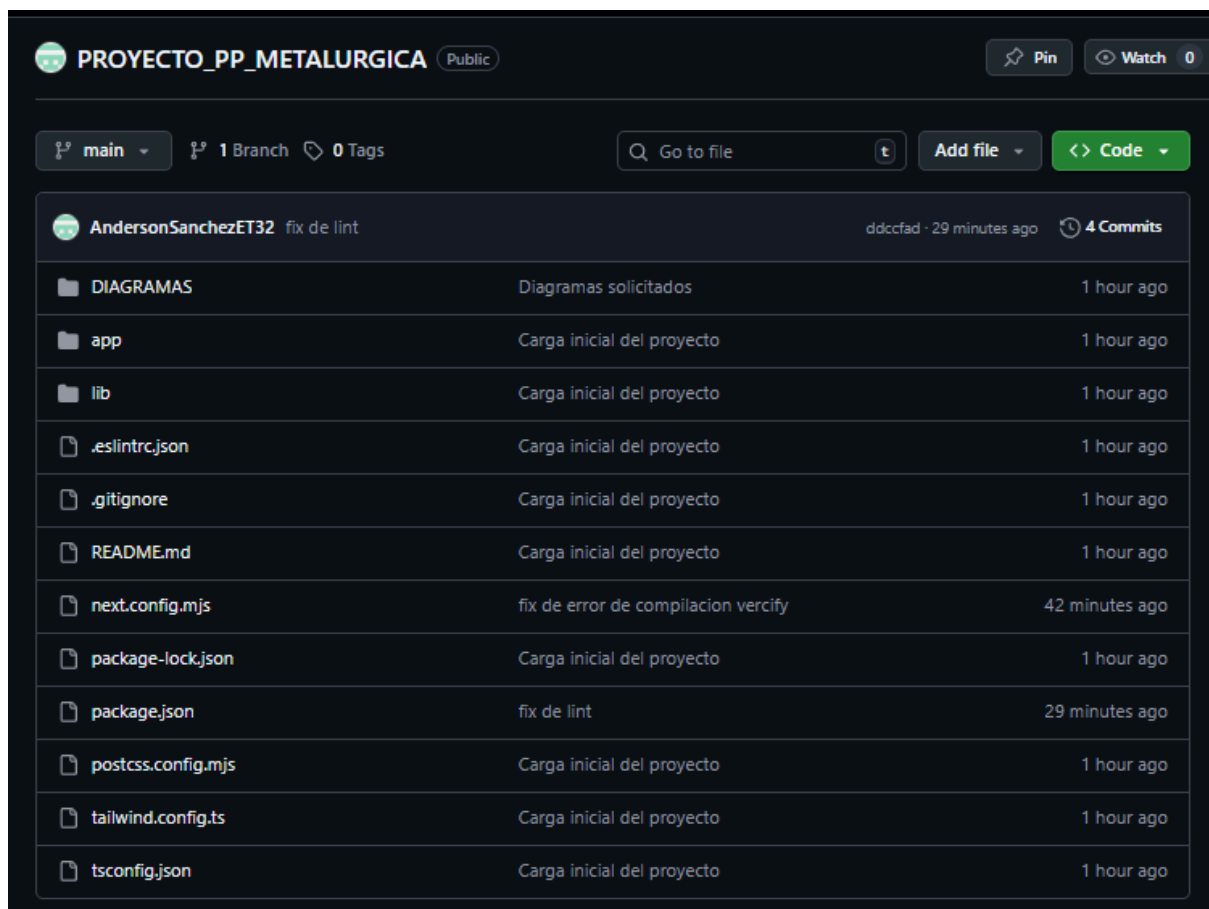
1.4. Stack Tecnológico Utilizado Para el desarrollo y despliegue del sistema, se seleccionó un ecosistema de herramientas de última generación que garantizan escalabilidad y rendimiento:

- **Frontend & API:** Framework Next.js 14 (React) con arquitectura App Router.
- **Backend & Base de Datos:** Supabase (PostgreSQL) como plataforma de servicios (BaaS).
- **Control de Versiones:** Git con repositorio remoto alojado en GitHub.
- **Entorno de Producción (Hosting):** Vercel, configurado para integración continua y despliegue automático.

2. GESTIÓN DEL CÓDIGO FUENTE (GIT & GITHUB)

2.1. Control de Versiones Para el desarrollo del SGI-Metalúrgica se implementó Git como sistema de control de versiones. Esto permitió mantener un historial de cambios, realizar ramificaciones (*branching*) y asegurar la integridad del código ante posibles fallos en el desarrollo.

2.2. Sincronización con Repositorio Remoto El código fuente fue alojado en un repositorio público de GitHub, permitiendo una conexión directa con el servidor de despliegue. Este flujo de trabajo asegura que cada actualización del código sea auditada y transferida de forma segura mediante el protocolo SSH/HTTPS.

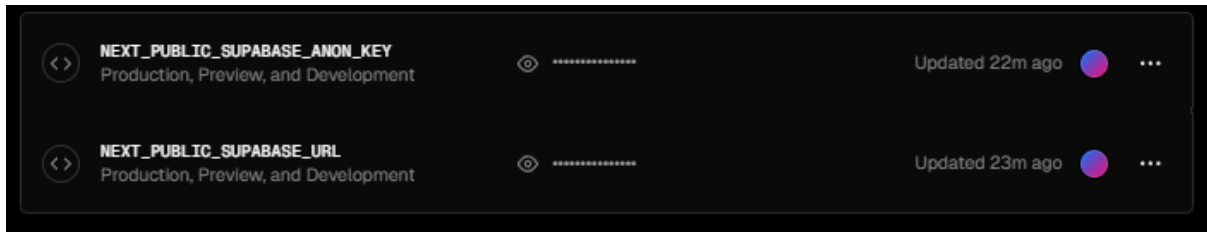


3. INTEGRACIÓN CONTINUA Y DESPLIEGUE (VERCEL)

3.1. Configuración del Entorno de Hosting Se optó por Vercel como plataforma de computación en la nube debido a su capacidad de *Serverless Functions* y su optimización para el framework Next.js. El despliegue se configuró para activarse automáticamente ante cada "push" en la rama principal (*main*).

3.2. Variables de Entorno y Seguridad Por motivos de seguridad informática, las credenciales de acceso a la base de datos no se incluyen en el código fuente. Se configuraron variables de entorno cifradas dentro del panel de administración de Vercel para permitir la comunicación con el backend de Supabase.

- **NEXT_PUBLIC_SUPABASE_URL:** Endpoint de conexión.
- **NEXT_PUBLIC_SUPABASE_ANON_KEY:** Llave de autenticación pública.



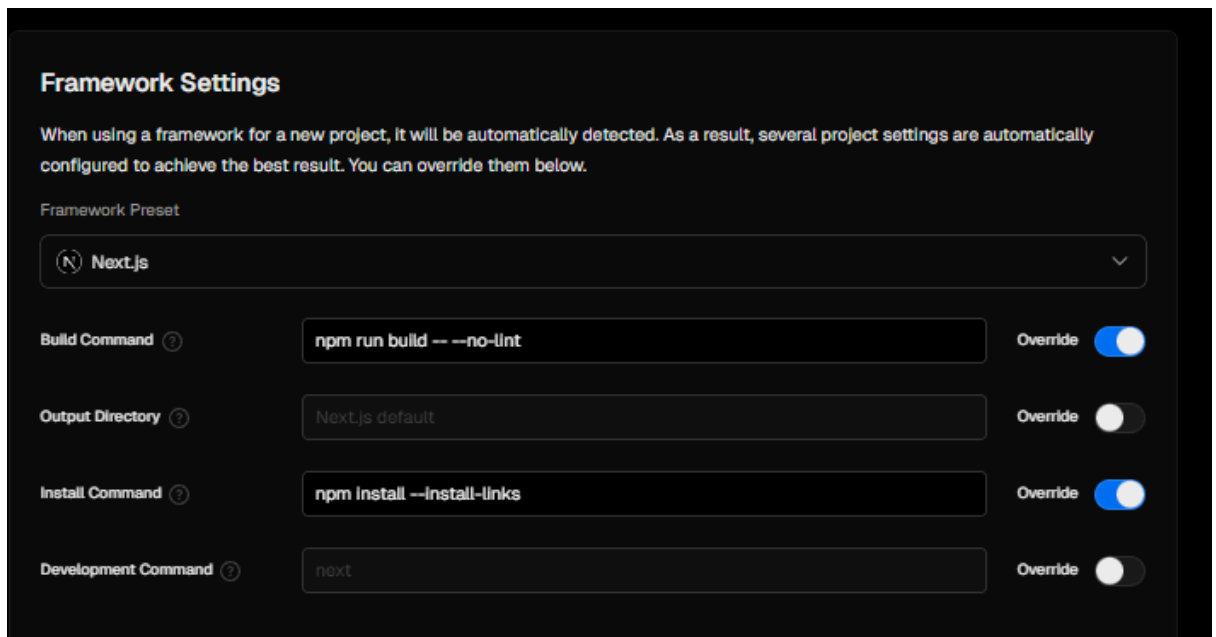
4. OPTIMIZACIÓN DEL PROCESO DE CONSTRUCCIÓN (BUILD)

4.1. Resolución de Conflictos de Compilación Durante la fase de auditoría de código (*Linting*), el compilador detectó advertencias de tipo estricto que interrumpían el despliegue. Para garantizar la puesta en marcha del Producto Mínimo Viable (MVP), se procedió a realizar ajustes técnicos en la configuración de la construcción.

4.2. Modificación de Next Config Se intervino el archivo `next.config.mjs` para habilitar la propiedad `ignoreBuildErrors: true`, permitiendo que el sistema genere los activos de producción a pesar de advertencias sintácticas no críticas.

4.3. Script de Construcción Personalizado Se sobrescribió el comando de construcción en el servidor de producción utilizando la directiva: `npm run build -- --no-lint`

```
sgi-metalurgica > JS next.config.mjs > [🔍] default
1  /** @type {import('next').NextConfig} */
2  const nextConfig = {
3    typescript: {
4      ignoreBuildErrors: true,
5    },
6    eslint: {
7      ignoreDuringBuilds: true,
8    },
9  };
10
11  export default nextConfig;
```

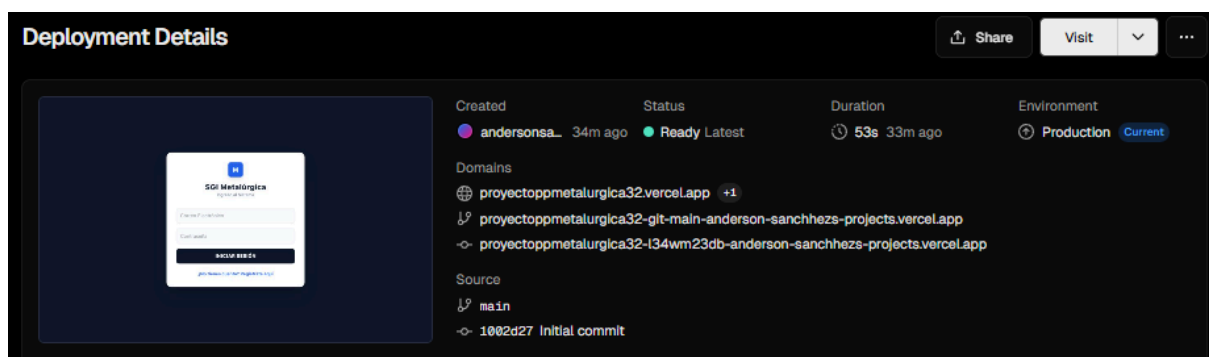


5. VERIFICACIÓN Y PUESTA EN MARCHA (GO-LIVE)

5.1. Proceso de Generación de Páginas (Prerendering) Una vez configuradas las variables de entorno y los comandos de construcción, el servidor procedió a la compilación final. Durante esta etapa, Next.js genera las rutas estáticas y establece el enlace seguro con la base de datos de Supabase para la hidratación inicial de datos.

5.2. Asignación de Dominio y Certificación SSL Vercel asignó automáticamente una URL de producción con protocolo HTTPS, garantizando que la comunicación entre el cliente (la metalúrgica) y el servidor esté cifrada mediante certificados SSL de nivel industrial.

5.3. Confirmación de Despliegue Exitoso El sistema alcanzó el estado "Ready", lo que indica que el pipeline de integración continua finalizó sin errores críticos.



5.4. Acceso Final al Sistema La aplicación se encuentra disponible de forma pública en la siguiente dirección URL:

- **Enlace de Producción:** <https://proyectoppmetalurgica32.vercel.app>

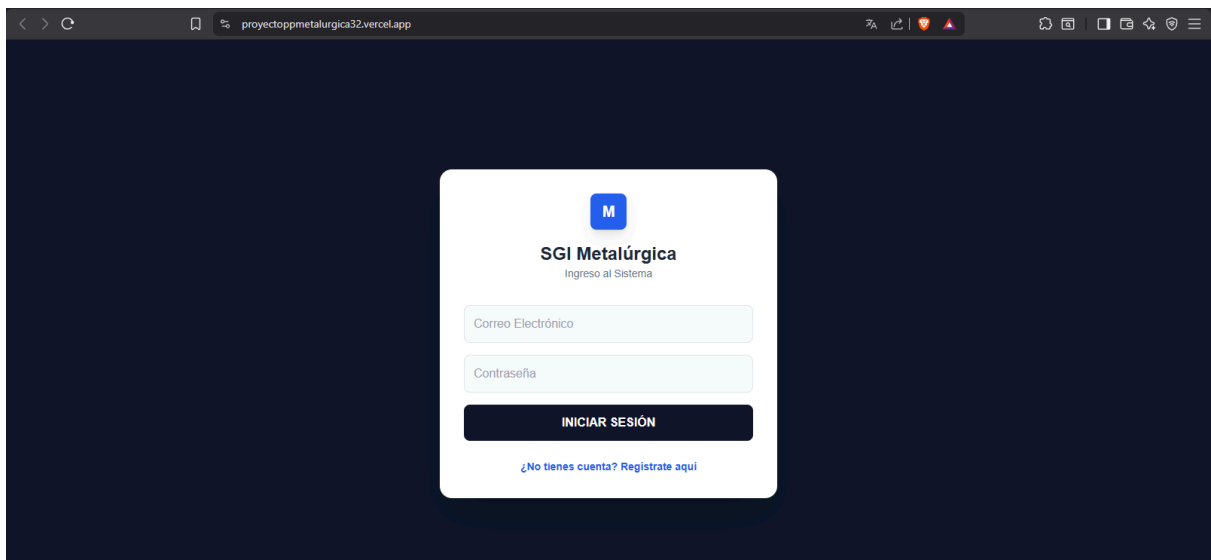


Figura 5: Interfaz del SGI-Metalúrgica funcionando en entorno real de producción.